Name: M. Sahasra

Reg. No.: 192425317

Course Code: ITA0403

Course Name: Statistics with R Programming

Faculty Name: Jaithunbi A K

Slot: A

LAB OBSERAVTION

8) Generate random numbers from a normal distribution; count occurrences.

Answer)

Code:

```
set.seed(10)
# Generate 100 random numbers
numbers <- rnorm(100, mean = 50, sd = 10)
# Round numbers to integers
rounded_numbers <- round(numbers)
# Count occurrences
frequency <- table(rounded_numbers)
print("First 20 Generated Values:")
print(head(numbers, 20))
print("Frequency of Rounded Values:")
print(frequency)
```

Output:

```
> source("D:/WORD DOCUMENTS/R Programming/Lab/Programs/Experiment-8 Generate Random Numbers from a Normal Distribution.R")
[1] "First 20 Generated Values:"
 [1] 50.18746 48.15747 36.28669 44.00832 52.94545 53.89794 37.91924 46.36324 33.73327 47.43522 61.01780 57.55782 47.61766 59.87445
[15] 57.41390 50.89347 40.45056 48.04850 59.25521 54.82979
[1] "Frequency of Rounded Values:"
rounded_numbers
28 29 31 32 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 64 65 71 72
 1  1  1  1  2  2  5  1  4  1  3  2  3  4  3  2  5  5  6  3  2  3  3  3  4  3  3  6  3  2  2  3  4  1  1  1  1
> |
```

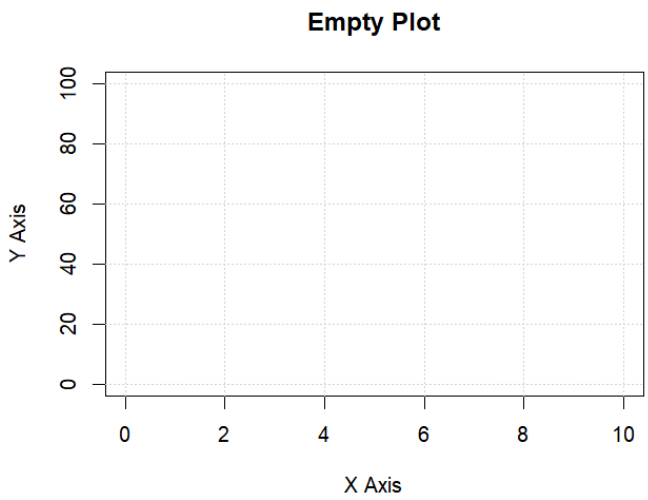
9) Create empty plots with specified axis limits.

Answer)

Code:

```
# Create an empty plot
plot(
  x = NA,
  y = NA,
  xlim = c(0, 10),
  ylim = c(0, 100),
  main = "Empty Plot",
  xlab = "X Axis",
  ylab = "Y Axis",
  type = "n"
)
# Add grid lines
grid()
```

Output:



10. Create and explore a data frame exam_data with name, score, attempts, and qualify fields. Perform extract, add row/column, sort, save to file.

Answer)

Code:

```
# Create the data frame
exam_data <- data.frame(
  name = c("Anu", "Bala", "Charan", "Divya", "Esha"),
  score = c(85, 72, 90, 65, 78),
  attempts = c(1, 2, 1, 3, 2),
  qualify = c(TRUE, TRUE, TRUE, FALSE, TRUE)
)
print("Original Data Frame:")
print(exam_data)
# Extract a row
print("First Row:")
print(exam_data[1, ])
# Extract a column
print("Student Names:")
print(exam_data$name)
# Add a new row
new_row <- data.frame(
  name = "Farhan",
  score = 88,
  attempts = 1,
  qualify = TRUE
)
exam_data <- rbind(exam_data, new_row)
# Add a new column
exam_data$grade <- ifelse(exam_data$score >= 80, "A", "B")
print("After Adding Row and Column:")
print(exam_data)
```

```
# Sort by score in descending order
exam_data <- exam_data[order(-exam_data$score), ]
print("Sorted Data Frame:")
print(exam_data)
# Save to CSV file
write.csv(exam_data, "exam_data.csv", row.names = FALSE)
cat("Data saved successfully to exam_data.csv\n")
```

Output:

```
> source("D:/WORD DOCUMENTS/R Programming/Lab/Programs/Experiment-10 Create and Explore the exam_data Data Frame.R")
[1] "Original Data Frame:"
    name score attempts qualify
1    Anu    85        1    TRUE
2   Bala    72        2    TRUE
3 Charan    90        1    TRUE
4  Divya    65        3   FALSE
5   Esha    78        2    TRUE
[1] "First Row:"
  name score attempts qualify
1  Anu    85        1    TRUE
[1] "Student Names:"
[1] "Anu"    "Bala"   "Charan" "Divya"  "Esha"
[1] "After Adding Row and Column:"
    name score attempts qualify grade
1    Anu    85        1    TRUE     A
2   Bala    72        2    TRUE     B
3 Charan    90        1    TRUE     A
4  Divya    65        3   FALSE     B
5   Esha    78        2    TRUE     B
6 Farhan    88        1    TRUE     A
[1] "Sorted Data Frame:"
    name score attempts qualify grade
3 Charan    90        1    TRUE     A
6 Farhan    88        1    TRUE     A
1    Anu    85        1    TRUE     A
5   Esha    78        2    TRUE     B
2   Bala    72        2    TRUE     B
4  Divya    65        3   FALSE     B
Data saved successfully to exam_data.csv
>
```

11. Write an R program to read a .csv file and display contents.

Answer)

Code:

```
# Create a sample CSV file
student_data <- data.frame(
  Name = c("Anu", "Bala", "Charan"),
  Marks = c(85, 90, 78),
  Department = c("CSE", "IT", "ECE")
)
write.csv(student_data, "students.csv", row.names = FALSE)
```

```
# Read the CSV file
data <- read.csv("students.csv")
# Display contents
print("CSV File Contents:")
print(data)
```

Output:

```
> source("D:/WORD DOCUMENTS/R Programming/Lab/Programs/Experiment-11 Read a CSV File and Display Its Contents.R")
[1] "CSV File Contents:"
    Name Marks Department
1    Anu    85        CSE
2   Bala    90         IT
3 Charan    78        ECE
>
```

12. Perform data reshaping on airquality dataset: melt, cast, compute monthly averages for Ozone, Solar.R, Wind, and Temperature.

Answer)

Code:

```
# Install the package once if required
# install.packages("reshape2")
library(reshape2)
# Load built-in dataset
data("airquality")
print("First Six Rows:")
print(head(airquality))
# Melt the dataset
melted_data <- melt(
  airquality,
  id.vars = c("Month", "Day"),
  measure.vars = c("Ozone", "Solar.R", "Wind", "Temp"),
  variable.name = "Measurement",
  value.name = "Value"
)
print("Melted Data:")
print(head(melted_data))
```

```
# Cast the data to calculate monthly averages
monthly_avg <- dcast(
  melted_data,
  Month ~ Measurement,
  fun.aggregate = function(x) mean(x, na.rm = TRUE)
)
print("Monthly Averages:")
print(monthly_avg)
```

Output:

```
> source("D:/WORD DOCUMENTS/R Programming/Lab/Programs/Experiment-12 Data Reshaping Using the airquality Dataset.R")
[1] "First Six Rows:"
  Ozone Solar.R Wind Temp Month Day
1    41     190  7.4   67     5   1
2    36     118  8.0   72     5   2
3    12     149 12.6   74     5   3
4    18     313 11.5   62     5   4
5    NA      NA 14.3   56     5   5
6    28      NA 14.9   66     5   6
[1] "Melted Data:"
  Month Day Measurement Value
1     5   1       Ozone    41
2     5   2       Ozone    36
3     5   3       Ozone    12
4     5   4       Ozone    18
5     5   5       Ozone    NA
6     5   6       Ozone    28
Using Value as value column: use value.var to override.
[1] "Monthly Averages:"
  Month    Ozone  Solar.R      Wind     Temp
1     5 23.61538 181.2963 11.622581 65.54839
2     6 29.44444 190.1667 10.266667 79.10000
3     7 59.11538 216.4839  8.941935 83.90323
4     8 59.96154 171.8571  8.793548 83.96774
5     9 31.44828 167.4333 10.180000 76.90000
>
```

13. Combine multiple arrays row-wise.

Answer)

Code:

```
# Create two matrices
arr1 <- matrix(1:6, nrow = 2, ncol = 3, byrow = TRUE)
arr2 <- matrix(7:12, nrow = 2, ncol = 3, byrow = TRUE)
print("First Array:")
print(arr1)
print("Second Array:")
print(arr2)
# Combine row-wise
```

combined_array <- rbind(arr1, arr2)

print("Combined Array:")

print(combined_array)

Output:

```
> source("D:/WORD DOCUMENTS/R Programming/Lab/Programs/Experiment-13 Combine Multiple Arrays Row-Wise.R")
[1] "First Array:"
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[1] "Second Array:"
     [,1] [,2] [,3]
[1,]    7    8    9
[2,]   10   11   12
[1] "Combined Array:"
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
[4,]   10   11   12
>
```

14. Explore and manipulate ChickWeight dataset (sorting, melting, casting by Diet)

Answer)

Code:

Install the package once if required

install.packages("reshape2")

library(reshape2)

Load dataset

data("ChickWeight")

print("First Six Rows:")

print(head(ChickWeight))

print("Dataset Structure:")

str(ChickWeight)

Sort by weight

sorted_data <- ChickWeight[order(ChickWeight$weight),]

print("First Six Rows Sorted by Weight:")

print(head(sorted_data))

Melt the dataset

melted_data <- melt(

  ChickWeight,

```
  id.vars = c("Chick", "Diet", "Time"),
  measure.vars = "weight",
  variable.name = "Measurement",
  value.name = "Weight"
)
print("Melted Data:")
print(head(melted_data))
# Calculate average weight by Time and Diet
diet_summary <- dcast(
  melted_data,
  Time ~ Diet,
  value.var = "Weight",
  fun.aggregate = mean
)
print("Average Weight by Time and Diet:")
print(diet_summary)
```

Output:

```
> source("D:/WORD DOCUMENTS/R Programming/Lab/Programs/Experiment-14 Explore and Manipulate the ChickWeight Dataset.R")
[1] "First Six Rows:"
  weight Time Chick Diet
1     42    0     1    1
2     51    2     1    1
3     59    4     1    1
4     64    6     1    1
5     76    8     1    1
6     93   10     1    1
[1] "Dataset Structure:"
Classes ‘nfnGroupedData’, ‘nfGroupedData’, ‘groupedData’ and 'data.frame':      578 obs. of  4 variables:
 $ weight: num  42 51 59 64 76 93 106 125 149 171 ...
 $ Time  : num  0 2 4 6 8 10 12 14 16 18 ...
 $ Chick : Ord.factor w/ 50 levels "18"<"16"<"15"<..: 15 15 15 15 15 15 15 15 15 15 ...
 $ Diet  : Factor w/ 4 levels "1","2","3","4": 1 1 1 1 1 1 1 1 1 1 ...
 - attr(*, "formula")=Class 'formula'  language weight ~ Time | Chick
  .. ..- attr(*, ".Environment")=<environment: R_EmptyEnv>
 - attr(*, "outer")=Class 'formula'  language ~Diet
  .. ..- attr(*, ".Environment")=<environment: R_EmptyEnv>
 - attr(*, "labels")=List of 2
  ..$ x: chr "Time"
  ..$ y: chr "Body weight"
 - attr(*, "units")=List of 2
  ..$ x: chr "(days)"
  ..$ y: chr "(gm)"
[1] "First Six Rows Sorted by Weight:"
    weight Time Chick Diet
196     35    2    18    1
26      39    2     3    1
195     39    0    18    1
293     39    0    27    2
305     39    0    28    2
317     39    0    29    2
```

```
[1] "Melted Data:"
  Chick Diet Time Measurement Weight
1     1    1    0      weight     42
2     1    1    2      weight     51
3     1    1    4      weight     59
4     1    1    6      weight     64
5     1    1    8      weight     76
6     1    1   10      weight     93
[1] "Average Weight by Time and Diet:"
   Time         1     2     3        4
1     0  41.40000  40.7  40.8  41.0000
2     2  47.25000  49.4  50.4  51.8000
3     4  56.47368  59.8  62.2  64.5000
4     6  66.78947  75.4  77.9  83.9000
5     8  79.68421  91.7  98.4 105.6000
6    10  93.05263 108.5 117.1 126.0000
7    12 108.52632 131.3 144.4 151.4000
8    14 123.38889 141.9 164.5 161.8000
9    16 144.64706 164.7 197.4 182.0000
10   18 158.94118 187.7 233.1 202.9000
11   20 170.41176 205.6 258.9 233.8889
12   21 177.75000 214.7 270.3 238.5556
> 
```